

动态规划入门 线性动态规划

主讲：黄凤林



→ 动态规划

➤ 前言

➤ 求解这样一个问题

某人要从 S_1 前往 S_N ，其中 S_i 至 S_{i+1} 有若干种行进方式（步行，自行车，公交车，越野车，etc），分别要花费不同的时间。

问从 S_1 走到 S_N 最快到达的时间是多少？



→ 动态规划

- 要求出从 S_1 到 S_N 的最少时间，则我们需要先求出从 S_1 到 S_{N-1} 的最少时间，然后从 S_{N-1} 到 S_N 再选择一个最优的行进方式，从而使得从 S_1 到 S_N 花费最少的时间。
- 要求解这个问题的最优解，我们发现是一个**多阶段**的问题，需要从 S_1 出发，依次求出到 S_2 、 S_3 、 $S_4 \dots S_{N-1}$ 、 S_N 的最优解；在**每一个 S_i 到 S_{i+1} 又需要选择最优的行进方式**。所以这就是一个**典型的多阶段决策性最优解问题**，这类问题的最佳方法就是动态规划。



→ 动态规划

- 求解这个问题的一般做法是：
- 我们用 $dp[i]$ 表示从 S_1 到达 S_i 地花费的最少时间，则有
- $dp[i] = dp[i-1] + \text{S}_{i-1} \text{到达} \text{S}_i \text{最优行进方式花费的时间}$
- 边界条件为 $dp[1] = 0$
- 最后的答案就是 $dp[n]$
- 其实这个求解方法就是动态规划的方式
- 其中 a_i ：**表示阶段，也表示到达 i 地的状态（因为这里只需要一个状态）。**
- S_{i-1} 到达 S_i 最优行进方式花费的时间**表示决策。**

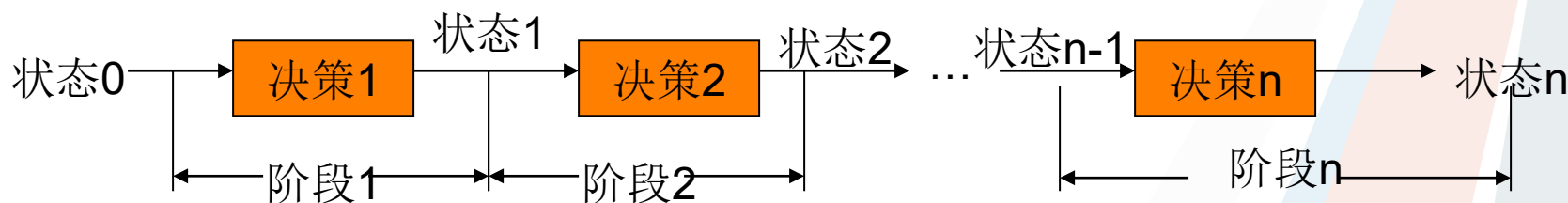


→ 动态规划

- **动态规划**(Dynamic Programming)是**研究多阶段决策过程最优化问题的一种数学方法**，英文缩写DP。
- 在动态规划中，**为了寻找一个问题的最优解（即最优决策过程），将整个**问题划分成若干个相应的阶段，并在每个阶段都根据先前所作出的决策作出当前阶段最优决策，进而得出整个问题的最优解。
- 能采用动态规划求解的问题的一般要具有三个性质：
- **最优化原理**：如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构，即满足最优化原理。
- **无后效性**：即某阶段状态一旦确定，就不受这个状态以后决策的影响。也就是说，某状态以后的过程不会影响以前的状态，只与当前状态有关。
- **有重叠子问题**：即子问题之间是不独立的，一个子问题在下一阶段决策中可能被多次使用到。（该性质并不是动态规划适用的必要条件，但是如果没有这条性质，动态规划算法同其他算法相比就不具备优势）

→ 动态规划

在现实生活中，有一类活动的过程，由于它的特殊性，可将过程分成若干个相互联系阶段，在它的每一个阶段都需要作出决策，从而使整个问题达到最好的活动效果。当然各个阶段决策的选取不是任何确定的，它依赖于当前面临的状态，又影响以后的发展，各个阶段决策确定后，就组成一个决策序列，因而也就确定了整个过程一条活动路线，这种把一个问题看作是一个前后关联具有链状结构的多阶段过程就称为多阶段决策过程，这种问题就称为多阶段决策问题。如下图：





→ 动态规划

- ◆ 动态规划是求解多阶段决策性最优解问题的有利武器，对思维难度要求高，常见实现策略**递推和记忆化搜索**两种，唯有多练习题目才能提高。
- ◆ **动态规划根据元素间关系，可以划分为常见模型：**
- ◆ 线性动态规划（一维、二维）
- ◆ 坐标动态规划（二维、三维）
- ◆ 区间动态规划（区间范围内）
- ◆ 背包动态规划（9种背包）
- ◆ 树形动态规划（基于树上的记忆化搜索）
- ◆ 状压动态规划（数位、按位DP）
- ◆ **动态规划的优化（单调性、斜率、四边形）**



→ 动态规划的几个要点

- ◆ 主体思想:
同一件事情不做第二次(同一状态不会重复计算)
- ◆ 状态表示:
用问题的某些特征参数描述当前的问题
- ◆ 状态转移方程:
状态值之间的递推关系(计算关系)
边界条件
递推顺序
- ◆ 实现方式:
自顶向下(不好枚举可行决策): 记忆化的搜索形式
自底向上: 递推形式



→ 动态规划的适用范围

- **动态规划用于解决多阶段决策最优化问题**，但是不是所有的最优化问题都可以用动态规划解答呢？一般在题目中出现求最优解的问题就要考虑动态规划了，但是否可以用还要满足三个条件：
- **最优子结构（最优化原理）**
 - **无后效性**
 - **有重叠子问题**(即子问题之间是不独立的，一个子问题在下一阶段决策中可能被多次使用到)。



➔ 动态规划题目特点

- ◆ 可以使用动态规划的题目特点
 - ✓ 一个大问题可以划分为若干个子问题
 - ✓ 最优子结构
 - ✓ 子问题: 性质一样但是规模变小的问题
 - ✓ 在计算时子问题有重叠
 - ✓ 重叠子结构
- ◆ 动态规划高效的原理: 重复的子问题不重复处理



→ 动态规划解题步骤

➤ 动态规划问题的解题步骤：

1. 拆分问题，划分阶段
2. 根据阶段，确定各个阶段的状态
3. 根据阶段间的状态确定转移决策
4. 综合考虑写成状态转移方程
5. 根据状态转移方程特性，确定程序实现方法
(递推或记忆化递归)
6. 确定初始状态、边界条件及其他注意和遗漏情况



➔ 动态规划模型---线性模型

特点：数据元素之间构成前后关系(线性关系)

- 基本思想： $dp[i]$ 表示到 i 这个阶段的最优解
- **动态规划用维数来维护更多的状态和特征**
- $dp[i][j], dp[i][j][k], \dots$

例2：求最短路(2)

某人要从 S_1 前往 S_N ，其中 S_i 至 S_{i+1} 有若干种行进方式（步行，自行车，公交车，越野车，etc），分别要花费一定的时间，并且如果在一个地点选择切换行进方式，需要额外消耗一定的时间。

问最快到达的时间是多少？

→ 问题分析

- 划分阶段、选择状态：
- 使用与上面一样的方案发现不可行，无法解决判定是否需要切换行进方式
- 加一维状态进行表示
- 用 $dp[i][j]$ 表示要从 S_1 到达 S_i ，在 S_i 时使用的出行方式为 j ，所需最短的时间
- 写出规划方程(包括边界条件)

$$dp[i][j] = \min(dp[i-1][j] + a[i-1][j], dp[i-1][k] + a[i-1][k] + b[k][j])$$

$a[i-1][j]$ ：表示从 S_{i-1} 到 S_i 用第 j 种行进方式花费的时间

$b[k][j]$ ：表示从 k 中行进方式换成 j 种行进方式花费的时间

K 是一个枚举行进方式的枚举变量



➔ 最长上升子序列

- 设有整数序列 $b_1, b_2, b_3, \dots, b_m$ ，若存在下标 $i_1 < i_2 < i_3 < \dots < i_n$ ，且 $b_{i_1} < b_{i_2} < b_{i_3} < \dots < b_{i_n}$ ，则称 $b_1, b_2, b_3, \dots, b_m$ 中有长度为 n 的上升子序列 $b_{i_1}, b_{i_2}, b_{i_3}, \dots, b_{i_n}$ 。
- 求序列 $b_1, b_2, b_3, \dots, b_m$ 中所有长度(n)最大的上升子序列
- 输入：整数序列。
- 输出：最大长度 n 和所有长度为 n 的序列个数。

➔ 最长上升子序列

➤ 动态规划求法

1. 设 $dp[i][j]$ 前 i 个数字以第 j 个数结尾的最大上升子序列长度，则： $dp[i][j] = \max\{dp[i-1][k] + 1\} (1 \leq k < j \leq m, b_j < b_i)$

边界 $dp[1][1] = 1$; 答案即为 $ans = \max(dp[m][k])$

由于在求 $dp[i][k]$ 时，除了 $dp[i][i]$ 需要从新求以外，其他 $dp[i][k]$ 与 $dp[i-1][k]$ 答案相同，因此可以优化维数。

2. 设 $dp[i]$ 为前 i 个数中以第 i 个数结尾的最大上升子序列长度，则 $dp[i] = \max\{dp[j] + 1\} (1 \leq j < i \leq m, b_j < b_i)$;

边界为 $dp[1] = 1$ ，最后 $ans = dp[m][k] (1 \leq k \leq m)$

→ 最长上升子序列

3. 设 $t[i]$ 为前 i 个数中以第 i 个数结尾的最长上升子序列的个数, 则
 $t[i] = \sum t[j] (1 \leq j < i \leq m, b_j < b_i, dp[i] = dp[j] + 1)$ 初始为 $t[i] = 1$; 当
 $dp[i] = \text{最大长度}$ 时, 将 $t[i]$ 累加

举例: 1 2 3 4 6 5 8 10 9

dp: 1 2 3 4 5 5 6 7 7

t: 1 1 1 1 1 1 2 2 2

答案: $dp=7$ 时, 边界为 $\sum t=4$

```
dp[1]=t[1]=1;
for(int i=2;i<=n;i++){
    for(int j=1;j<i;j++){
        if(a[i]>a[j]){
            if(dp[i]<dp[j]+1){
                dp[i]=dp[j]+1;t[i]=t[j];
            } else if(dp[i]==dp[j]+1){
                t[i]+=t[j];
            }
        }
    }
}
```


➔ 最长上升子序列 (升级)

➤ Hdu 5256

题意：给出一个数列， $a_1, a_2, a_3, a_4, \dots, a_n$ ，现在需要修改尽量少的元素，使得这个序列严格递增。

➤ **问题分析：**答案是否就是对原序列求一个最长上升子序列长度，然后用总长度-最长上升子序列长度呢？

答案是否定的，如：1 2 4 3 4 5这个序列的最长上升子序列长度为5，则修改最少元素为1个，明显是错误的。这里显然需要修改4个。

➤ **那这里应该怎么做呢？**

➔ 最长上升子序列 (升级)

- ◆ **问题分析：透过表面看本质**
- ◆ 因为 $a[i] < a[i+1]$
- ◆ 又因为 $a[i], a[i+1]$ 均为整数
- ◆ 所以 $a[i+1] - 1 \geq a[i]$
- ◆ 变形得 $a[i+1] - (i+1) \geq a[i] - i$
- ◆ 令 $b[i] = a[i] - i$;
- ◆ 所以可以求出 $b[i]$ 的最长不下降子序列
- ◆ 再用 $n - \text{len}$ 即为需要改变的最少个数



→ 青蛙过河

➤ NOIP2005过河(青蛙过河)

在河上有一座独木桥，一只青蛙想沿着独木桥从河的一侧跳到另一侧。在桥上有一些石子，青蛙很讨厌踩在这些石子上。由于桥的长度和青蛙一次跳过的距离都是正整数，我们可以把独木桥上青蛙可能到达的点看成数轴上的一串整点： $0, 1, \dots, L$ （其中 L 是桥的长度）。坐标为 0 的点表示桥的起点，坐标为 L 的点表示桥的终点。

青蛙从桥的起点开始，不停的向终点方向跳跃。一次跳跃的距离是 S 到 T 之间的任意正整数（包括 S, T ）。当青蛙跳到或跳过坐标为 L 的点时，就算青蛙已经跳出了独木桥。

题目给出独木桥的长度 L ，青蛙跳跃的距离范围 S, T ，桥上石子的位置。你的任务是确定青蛙要想过河，最少需要踩到的石子数。

对于30%的数据， $L \leq 10000$ ；

对于全部的数据， $L \leq 10^9$ ， $1 \leq S \leq T \leq 10$ ， $1 \leq M \leq 100$



➔ 问题分析

本题动态规划思路很简单，
用 $dp[i]$ 表示到达 i 位置踩到的最少石头数量，然后分两种情况：

如果 i 处有石头，则： $dp[i] = \max(dp[i], dp[j] + 1)$ $i - t \leq j \leq i - s$

如果 i 处没石头，则： $dp[i] = \max(dp[j])$ $i - t \leq j \leq i - s$

本题难点在于， $L = 10^9$ ，如果直接这样子做，显然会超时和超空间，所以我们要进行优化。

思考，虽然 L 很大，但是 M (石头数量)很小，因此会存在石头分布很稀疏的情况(中间很长一段距离都没石头)，这对解题有何帮助呢？我们来分析一下



→ 问题分析

➤ 第一种情况：当 $s=t$ 时

很简单，青蛙踩到的石头肯定是 s 的倍数，那么只要统计一下所有石子中有多少是 s 的倍数，输出即可。

➤ 第二种情况：当 $s < t$ 时，我们先来看一组数据。 $s=4$ ， $t=5$ 。

从数据中我们看到，12以后的点全部都是可以到达的了。如果 $s=4$ ， $t=5$ ，在一段100000的距离中没有石头，其实12以后的点都是不用递推就知道肯定能到达的。那么我们用原始的方法做就会浪费很大的资源。

所以当 $s=4$ ， $t=5$ 时，如果一段没有石头的区间长度在 $4*5=20$ 以外，那么我们只要递推前20就可以了，因为20后面的情况是一样的（仔细想想为什么？）。

假设 $s=3$ ， $t=5$ ，那么 $11=3+4+4$ 就也可以到达了。所以，只有当 $t=s+1$ 时，连续的点的起始位置才能尽量后面。最坏情况就是 $s=9$ ， $t=10$ 了（仔细想想为什么？）。

跟前面的 $s=4$ ， $t=5$ 的情况一样，其实 $s=9$ ， $t=10$ 时只要一段没有石头的区间长度在90之外，我们都把它当做90对待就可以了。

因此，我们每次对于一段没有石头的区间长度为 x ，如果 $x \leq t(t-1)$ ，我们仍然把它当做 x 来处理；相反，当 $x > t(t-1)$ 时，我们就把它当做 $t(t-1)$ 处理。这样，最大的复杂度就是 $t(t-1) * (\text{石头个数} + 1) = 90 * 101 = 9090$ ，比之前的复杂度大大降低。



参考代码

```
if(s==t){//s=t单独讨论
    int ans=0;
    for(int i=1;i<=m;i++) if(a[i]%s==0)ans++;
    cout<<ans;
    return 0;
}
sort(a,a+m+1);
a[m+1]=1;
for(int i=0;i<=m;i++)if(a[i+1]-a[i]>90)
    a[i+1]=a[i]+(a[i+1]-a[i])%90;//压缩路径
for(int i=1;i<=m;i++)sz[a[i]]=1;
for(int i=s;i<=t;i++){if(sz[i])f[i]=1;else f[i]=0;}
for(int i=2*s;i<=a[m+1];i++)
{
    for(int j=s;j<=t;j++)
    {
        if(j>i)break;
        f[i]=min(f[i-j],f[i]);
    }
    if(sz[i])f[i]++;
}
cout<<f[a[m+1]]<<endl;
```



➔ 护卫队

➤ 问题描述

护卫车队在一条单行的街道前排成一队，前面河上是一座单行的桥。因为街道是一条单行道，所以任何车辆都不能超车。桥能**承受一个给定的最大承载量**。

为了控制桥上的交通，桥两边各站一个指挥员。护卫车队被分成几个组，**每组中的车辆都能同时通过该桥**。当一组车队到达了桥的另一端，该端的指挥员就用电话通知另一端的指挥员，这样**下一组车队才能开始通过该桥**。**每辆车的重量是已知的，任何一组车队的重量之和不能超过桥的最大承重量**。被分在同一组的每一辆车都以其最快的速度通过该桥。**一组车队通过该桥的时间是用该车队中速度最慢的车通过该桥所需的时间来表示的**。

问题：计算出全部护卫车队通过该桥所需的最短时间值。

这里： $N \leq 1000$;



➔ 护卫队

➤ 问题分析

- 题面较长，其实质想表达的是**有一个长为n的车队，要在车辆顺序不变（不能超车）的情况下把它分为若干个小车队过桥，桥上只能有一个车队**，抽象地讲就是把一个数列分为若干份。问怎么分使得代价最优。
- 因此我们完全可以考虑用动态规划来解，动态规划的目的就是来找到分开车队的断点。
- **所以我们用 $dp[i]$ 表示前*i*辆车全部通过的最短时间，则 $dp[i] = \max(dp[i], dp[j-1] + st[j][i])$, $1 \leq j \leq i$**
 $st[j][i]$ 表示*j*到*i*组成一队，通过桥花费的最短时间。
DP方程表达的含义就是以*j*为断点，前*j*-1辆车通过的最优时间+[*j*与*i*]合成一队花费的时间。

→ 护卫队

```

int x,y;
scanf("%d%d",&maxv,&len,&n);
for(int i=1;i<=n;i++){
    scanf("%d",&x,&y);
    w[i]=w[i-1]+x;
    s[i]=len*60.00000/y;
}
dp[0]=0;
for (int i=1;i<=n;i++) dp[i]=999999999999999.0;
for(int i=1;i<=n;i++){
    double mint=-999999999999999.0;//mint表示车
    for(int j=i;j>=1;j--){
        if(w[i]-w[j-1]>maxv) break;//后面一队超
        else {
            mint=max(mint,s[j]);
            dp[i]=min(dp[i],dp[j-1]+mint);//f[i-
        }
    }
}
printf("%.11f\n",dp[n]);

```



➔ Print Article(HDU 3507)

- ◇ 题目大意：
- ◇ 给出N个单词，每个单词有个非负权值 C_i ，现要将它们分成连续的若干段，每段的代价为此段单词的权值和，还要加一个常数M，即 $(\sum C_i)^2 + M$ 。现在想求出一种最优方案，使得总费用之和最小。



问题分析

- ◆ 根据护卫队同样的思路，容易写出方程：
$$dp[i] = \min\{dp[j] + (s[i] - s[j])^2 + M\} (0 \leq j \leq i-1)$$
- ◆ 其中s是前缀和
- ◆ $dp[i]$ 表示输出到i的时候最少的花费。
可是范围是500000，又不能用单调队列
- ◆ 那怎么办呢？

问题分析

我们假设在求解 $dp[i]$ 时，存在 $j, k (j > k)$ 使得从 j 转移比从 k 转移更优，那么需要满足条件： $dp[j] + (s[i] - s[j])^2 + M < dp[k] + (s[i] - s[k])^2 + M$

展开上式：移项并消去再合并同类项得（**把 j 和 k 放到同一边**）
 $dp[j] - dp[k] + s[j]^2 - s[k]^2 < 2s[i](s[j] - s[k])$

把 $S[j] - S[k]$ 除过去，得到 $\frac{dp[j] - dp[k] + s[j]^2 - s[k]^2}{s[j] - s[k]} < 2s[i]$

我们设 $f[x] = dp[x] + s[x]^2$ ，就化成了 $\frac{f[j] - f[k]}{s[j] - s[k]} < 2s[i]$

即当且仅当 $(j > k)$ 且 $\frac{f[j] - f[k]}{s[j] - s[k]} < 2s[i]$ 时， j 对更新 $dp[i]$ 比 k 更新 $dp[i]$ 优。

休息一下qwq。这个东西好像**斜率**。

当一个数的 dp 值求完了，它的 f 值也跟着确定，我们就可以在空间中绘制出点 $(S[i], f[i])$ ，这个点代表已经求出 dp 值的一个点。

→ 问题分析

当我们要求解 $dp[t]$ 时，如果可用的集合里存在这样三个点，位置关系如图所示：

那么显然

$$\frac{f[j]-f[k]}{S[j]-S[k]} > \frac{f[i]-f[j]}{S[i]-S[j]}$$

这时候他们和 $2S[t+1]$ 的关系有3种：

$$\frac{f[j]-f[k]}{S[j]-S[k]} > \frac{f[i]-f[j]}{S[i]-S[j]} > 2S[t+1]$$

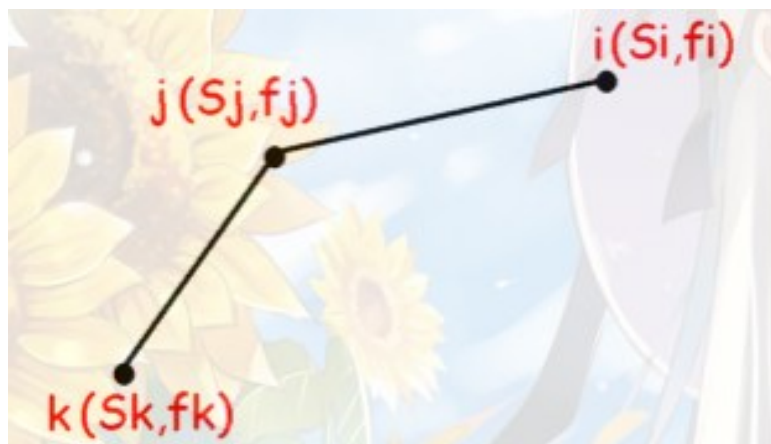
那么 j 比 i 优， k 比 j 优。

$$\frac{f[j]-f[k]}{S[j]-S[k]} > 2S[t+1] > \frac{f[i]-f[j]}{S[i]-S[j]}$$

那么 i 比 j 优， k 比 j 优。

$$2S[t+1] > \frac{f[j]-f[k]}{S[j]-S[k]} > \frac{f[i]-f[j]}{S[i]-S[j]}$$

那么 i 比 j 优， j 比 k 优。



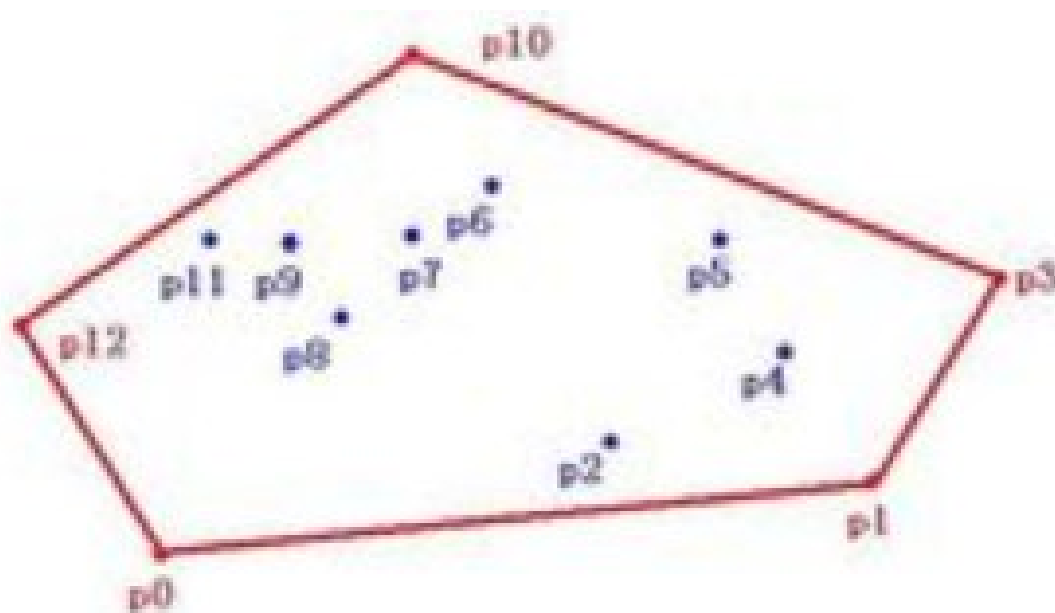
综上，不管什么样的 $S[t+1]$ ，从 j 转移都不会是最佳方案。

那么用一个数据结构维护一个凸包（下凸），每加入一个点就删去一些点，使其维持凸包的形态。最优转移一定在这个凸包中



→ 凸包

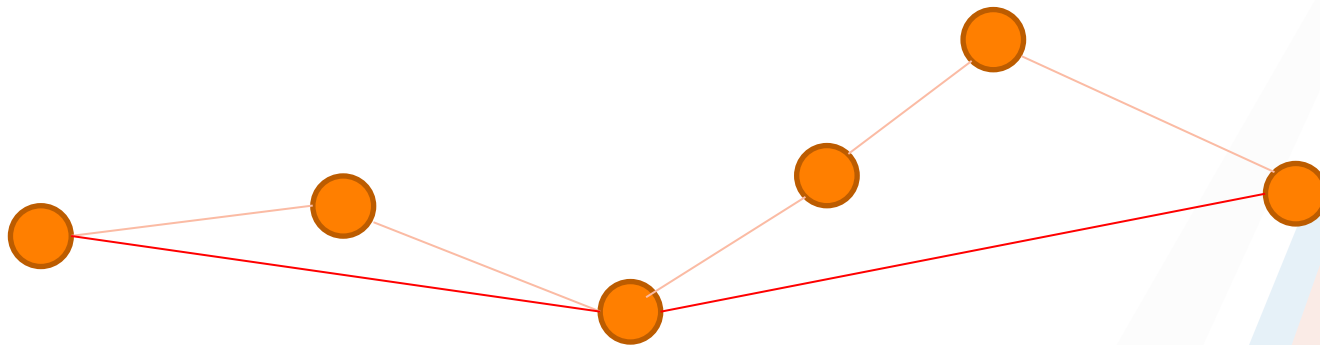
- 给定二维平面上的点集，凸包就是将最外层的点连接起来构成的凸多边形，它能包含点集中所有的点。





→ 凸包的求法

- 把平面上的点，按照x坐标从小到大排序，然后找出离y轴最近和最远的两个点，把凸包分成上凸壳和下凸壳两个部分，然后分别求。
- 下凸壳的求法：栈模拟



扩展分析

- 但还是不能 $O(1)$ 。在凸包里，谁又是最最优呢？
- 首先一定数据结构里的凸包一定会是这样的：



- 假设 j 的斜率 $> 2S[t+1]$ 且 k 的斜率 $< 2S[t+1]$ 从图形特点我们可以发现 j 点比所有比 k 小的点都优，比所有比 i 大的也优。所以对于我们二分查找斜率比 $2S[t+1]$ 小但编号最大的点，就是最优的转移点。由于 $S[i]$ 也满足单调性，我们还可以直接维护一个单调队列就能解决这个问题。
- 推广一下，如果不等式右侧不是像 $S[i+1]$ 这样的单调函数，二分就好了。

→ 单调队列的维护

- 第一步：找斜率比 $2s[t+1]$ 小但编号最大的点

```
while(L<R&&slop(q[L],q[L+1])<=sum[i]) L++;
```

- 第二步：维护保持下凸壳图形的性质

```
while(L<R&&slop(q[R-1],q[R])>slop(q[R],i)) R--;
```

```
double slop(int k,int j)
{
    return (double)(dp[j]+sum[j]*sum[j]-dp[k]-sum[k]*sum[k])/(double)(2*(sum[j]-sum[k]));
}
```

参考博客：<https://www.cnblogs.com/orzzz/p/7885971.html#4050706>



代码实现

```
for(int i=1;i<=n;i++)
{
    scanf("%lld",&sum[i]);
    sum[i]+=sum[i-1];
    if(sum[i]==sum[i-1]) i--,n--;
}
int L=1,R=1;
dp[0]=0;
for(int i=1;i<=n;i++)
{
    while(L<R&&slop(q[L],q[L+1])<=sum[i]) L++; // 查找斜率比sum[i]小的
    dp[i]=dp[q[L]]+(sum[i]-sum[q[L]])*(sum[i]-sum[q[L]])+k;
    while(L<R&&slop(q[R-1],q[R])>slop(q[R],i)) R--;
    dl[++r]=i;
}
printf("%lld\n",dp[n]);
```

→ 斜率优化dp总结

- 1、写出dp方程
- 2、观察dp方程是否中，推出dp[i]的值是否与枚举1—j转移而来，且由于i有关系，形如：
 $f[i] = \min \text{ or } \max \{f[j] + (x[i] - x[j])^2\}$ ，这样子的样式。
- 3、对dp方程进行变形，得出斜率表达式。
- 4、用斜率优化dp，代码实现。注意：求最小值是维护下凸壳，求最大值是维护而上凸壳。



→ Picnic Cows (HDU3045)

- 题目大意：
给出一个有 N ($1 \leq N \leq 400000$)个正数的序列，要求把序列分成若干组（可以打乱顺序），每组的元素个数不能小于 T ($1 \leq T \leq N$)。每一组的代价是每个元素与最小元素的差之和，总代价是每个组的代价之和，求总代价的最小值。



→ Picnic Cows

- **问题分析：**由于可以打乱序列顺序，又知道代价为组内每个元素与最小值差之和，故想到贪心，先将序列排序。排完序后，会发现和护卫队问题就很相似了，依然是把序列分成若干份，然后最后总体代价最优。所以我们依然可以用 $dp[i]$ 表示前 i 个数分成若干份的最优代价，则方程如下：
- $dp[i] = \min(dp[i], dp[j] + (a[j+1 \sim i] - \text{mink}))$ ($0 \leq j < i$)
- mink 表示 $j+1 \sim i$ 范围内的最小值，排序后即 $a[j+1]$ 。
- 上式变形得到： $dp[i] = \min(dp[j] + \text{sum}[i] - \text{sum}[j] - a[j+1] * (i - j))$
- $\text{sum}[i]$ 表示前 i 个元素之和(前缀和)
- 这个方程的思路就是枚举 j 不断地计算状态值更新答案。但是数据规模达到了40000，这种以 $O(n^2)$ 为绝对上界方法在这里明显行不通，怎么办？？？



斜率优化

- 变形得出斜率表达式为：
- $f[i] = dp[i] - \text{sum}[i] + a[i+1] * i$
- $\frac{f[j] - f[k]}{a[j+1] - a[k+1]} < i$



→ 线性类动态规划

- 摆渡车(NOIP2018 普及组第三题)
- 有 n 名同学要乘坐摆渡车从人大附中前往人民大学，第 i 位同学在第 t_i 分钟去等车。只有一辆摆渡车在工作，但摆渡车容量可以视为无限大。摆渡车从人大附中出发、把车上的同学送到人民大学、再回到人大附中（去接其他同学），这样往返一趟总共花费 m 分钟（同学上下车时间忽略不计）。
- 摆渡车要将所有同学都送到人民大学。凯凯很好奇，如果他能任意安排摆渡车出发的时间，那么这些同学的等车时间之和最小为多少呢？注意：摆渡车回到人大附后可以即刻出发。



➔ 练习题目(落谷)

- P4072
- P4360
- P2365
- CF311B
- 作业博客:
<https://www.cnblogs.com/Judge/p/9551035.html>



思考题：乌龟棋

小明过生日的时候，爸爸送给他一副乌龟棋当作礼物。

乌龟棋的棋盘是一行 N 个格子，每个格子上一个分数（非负整数）。棋盘第 1 格是唯一的起点，第 N 格是终点，游戏要求玩家控制一个乌龟棋子从起点出发走到终点。

乌龟棋中 M 张爬行卡片，分成 4 种不同的类型（ M 张卡片中不一定包含所有 4 种类型的卡片，见样例），每种类型的卡片上分别标有 1、2、3、4 四个数字之一，表示使用这种卡片后，乌龟棋子将向前爬行相应的格子数。游戏中，玩家每次需要从所有的爬行卡片中选择一张之前没有使用过的爬行卡片，控制乌龟棋子前进相应的格子数，每张卡片只能使用一次。游戏中，乌龟棋子自动获得起点格子的分数，并且在后续的爬行中每到达一个格子，就得到该格子相应的分数。玩家最终游戏得分就是乌龟棋子从起点到终点过程中到过的所有格子的分数总和。

很明显，用不同的爬行卡片使用顺序会使得最终游戏的得分不同，小明想要找到一种卡片使用顺序使得最终游戏得分最多。

现在，告诉你棋盘上每个格子的分数和所有的爬行卡片，你能告诉小明，他最多能得到多少分吗？



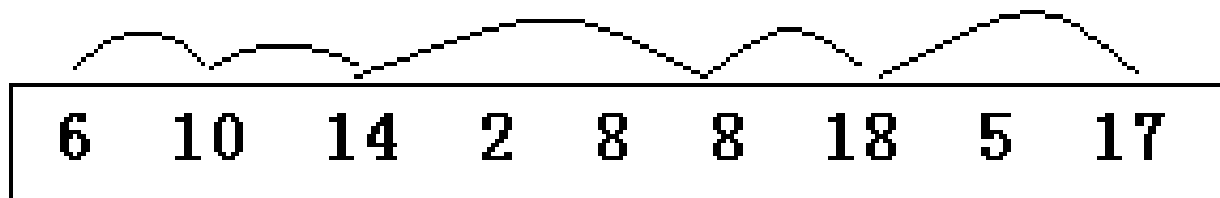
→ 样例

9 5

6 10 14 2 8 8 18 5 17

1 3 1 2 1

使用卡片的顺序为1 1 3 1 2，得到73分，如下图



$$6+10+14+8+18+17=73$$



→ 思考

- 1、阶段划分
- 2、阶段的状态
- 3、转移方程？
- 4、边界、细节处理

→ 状态转移方程

状态设计可以用四维 $dp[a][b][c][d]$ 来维护，表示含义如下 dp ["1"的张数]["2"的张数]["3"的张数]["4"的张数]的最优得分，方程就很显然了 $dp[a][b][c][d] = \max\{$

$$(a > 0) f[a-1][b][c][d] + a[a*1 + b*2 + c*3 + d*4],$$

$$(b > 0) f[a][b-1][c][d] + a[a*1 + b*2 + c*3 + d*4],$$

$$(c > 0) f[a][b][c-1][d] + a[a*1 + b*2 + c*3 + d*4],$$

$$(d > 0) f[a][b][c][d-1] + a[a*1 + b*2 + c*3 + d*4]$$

}

当然，由于第一个点自动得分，所 $dp[0][0][0][0] = a[1]$

(这也是为什么我们只用计算每一步终点的得分，可以确保起点已经计算)



思考题：不等数列

➤ 问题描述：

将1到n任意排列，然后在排列的每两个数之间根据他们的大小关系插入 “>” 和 “<” 。

问在所有排列中，有多少个排列恰好有k个 “<” 。

答案对2012取模。

样例：

5 2

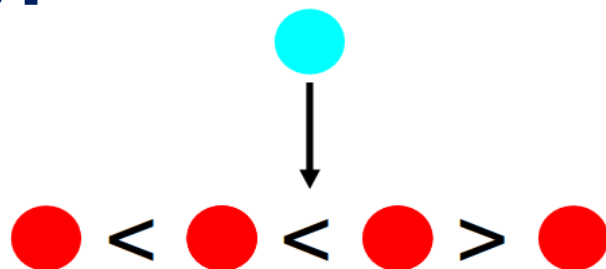
66



→ 思考

- 1、阶段划分
- 2、阶段的状态
- 3、转移方程?
- 4、边界、细节处理

→ 问题分析



蓝色是当前插入的数字,如果它插入到<关系的两个数字之间(或数列最左端),就会使数列的**<符号个数不变, >符号个数+1**



类似的, 插入到>关系的两个数字之间(或者数列最右端),
数列的<数量+1, >数量不变。

$dp[i][j]$ 表示前*i*个数字构成的数列中, 恰有*j*个<符号的方案数(>符号自然就有*i-j-1*个)。

$$dp[i][j] = dp[i-1][j-1] * (i-j) + dp[i-1][j] * (j+1)$$



➔ 线性结构模型二

特点：二条直线上数据元素互相关联

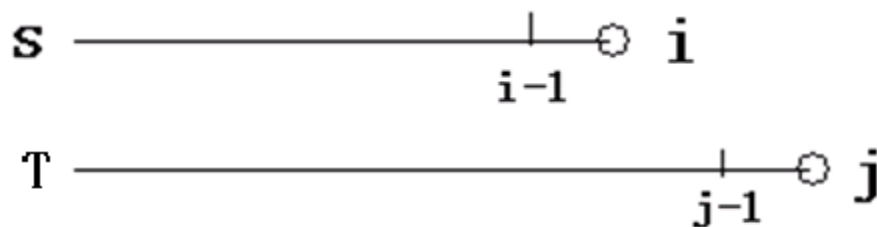


➔ 求最长公共子序列

- 给定的字符序列 $X = "x_0, x_1, \dots, x_{m-1}"$ ，序列 $Y = "y_0, y_1, \dots, y_{k-1}"$ 是 X 的子序列，存在 X 的一个严格递增下标序列 $\langle i_0, i_1, \dots, i_{k-1} \rangle$ ，使得对所有的 $j = 0, 1, \dots, k-1$ ，有 $x_{i_j} = y_j$ 。
- 例如， $X = "ABCB DAB"$ ， $Y = "BCDB"$ 是 X 的一个子序列。
- 给出两个字符串 $S1$ 和 $S2$ ，长度不超过5000。
- 求这两个串的最长公共子序列长度。

→ 求最长公共子序列

- ◆ 设 $f(i,j)$ 表示 S 的前 i 位与 T 的前 j 位的最长公共子串长度。则有，



$$f(i, j) = \begin{cases} 0, & \text{当 } i = 0 \parallel j = 0 \\ \max\{f(i, j-1), f(i-1, j)\}, & \text{当 } i, j > 0 \text{ 且 } s_i \neq t_j \\ f(i-1, j-1) + 1, & \text{当 } i, j > 0 \text{ 且 } s_i = t_j \end{cases}$$

- ◆ 时间复杂度 $O(n*m)$

→ 求最长公共子序列

	s	d	f	s	d	s
f	0	0	1	1	1	1
s	1	1	1	2	2	2
d	1	2	2	2	3	3
f	1	2	3	3	3	3
s	1	2	3	4	4	4

```
scanf("%s %s",&a,&b);
int lena,lenb;
lena=strlen(a); lenb=strlen(b);
for(int i=1;i<=lena;i++){
    for(int j=1;j<=lenb;j++){
        if(a[i-1]==b[j-1]) dp[i][j]=dp[i-1][j-1]+1;
        else dp[i][j]=max(dp[i-1][j],dp[i][j-1]);
    }
}
printf("%d\n",dp[lena][lenb]);
```



思考题：求最长公共子串

- 若一个字符串的一部分与另一个字符串的一部分相等，则称它为两个串的公共子串
- 给出两个串，求最长公共子串？
- 例如： $S1 = "123456"$, $S2 = "2346"$, 则 $S1$ 和 $S2$ 公共子串为 $"234"$ 。

→ 分析

- 考虑 A_i 和 B_j 是否相等,
- 如果 $A_i = B_j$, 则 $dp[i][j] = dp[i-1][j-1] + 1$,
- 如果 $A_i \neq B_j$, 则 $dp[i][j] = 0$;

	s	d	f	s	d	s
f	0	0	1	0	0	0
s	1	0	0	2	0	1
d	0	2	0	0	3	0
f	0	0	3	0	0	0
s	1	0	0	4	0	0



→ 最小编辑距离

字符串是数据结构和计算机语言里很重要的数据类型，在计算机语言中，对于字符串我们有很多的操作定义，因此我们可以对字符串进行很多复杂的运算和操作。实际上，所有复杂的字符串操作都是由字符串的基本操作组成。例如，把子串**a** 替换为子串**b**，就是用查找、删除和插入这三个基本操作实现的。因此，在复杂字符串操作的编程中，为了提高程序中字符操作的速度，我们就应该用最少的基本操作完成复杂操作。在这里，假设字符串的基本操作仅为：删除一个字符、插入一个字符和将一个字符修改成另一个字符这三种操作。我们把进行了一次上述三种操作的任意一种操作称为进行了一步字符基本操作。下面我们定义两个字符串的编辑距离：对于两个字符串**a**和**b**，通过上述的基本操作，我们可以把**a**变成**b**或**b**变成**a**；那么，把字符串**a**变成字符串**b**需要的最少基本字符操作步数称为字符串**a**和字符串**b** 的编辑距离。

例如，如 $a = \text{"ABC"}$ ， $b = \text{"CBCD"}$ ，则 a 与 b 的编辑距离为 2。
你的任务：编一个最快的程序来计算任意两个字符串的编辑距离。



→ 最小编辑距离

输入数据：

第 1 行为字符串 **a**；第 2 行为字符串 **b**。

注：字符串的长度不大于 1000，字符串中的字符全为大写字母。

输出数据：

编辑距离。

样例：

输入文件名：edit.in

ABC

CBCD

输出文件名：edit.out

2



→ 分析

与公共子串类似，考虑 A_i 是否等于 B_j

如 $A_i = B_j$,则当前不需要增加步骤，

$$dp[i][j] = dp[i-1][j-1]$$

如 $A_i \neq B_j$,则需要增加一个步骤，但是我们应该选取三种方式中最优的一种，所以得到：

$$dp[i][j] = \text{MIN} \begin{cases} dp[i-1][j] + 1, \text{ 插入操作} \\ dp[i-1][j-1] + 1, \text{ 修改操作} \\ dp[i][j-1] + 1, \text{ 删除操作} \end{cases}$$



子串

描述

有两个仅包含小写英文字母的字符串 A 和 B。

现在要从字符串 A 中取出 k 个互不重叠的非空子串，然后把这 k 个子串按照其在字符串 A 中出现的顺序依次连接起来得到一个新的字符串。请问有多少种方案可以使得这个新串与字符串 B 相等？

注意：子串取出的位置不同也认为是不同的方案。

提示： $n \leq 1000, m \leq 200$



→ 分析

设 $f[i][j][k]$ 为串 **A** 匹配到第 i 个位置(位置从1开始),串 **B** 匹配到 j 个位置而且恰好选串 **A** 第 i 个字符的方案总数.设 $s[i][j][k]$ 为串 **A** 匹配到第 i 个位置(位置从1开始),串 **B** 匹配到 j 个位置,的方案总数。

可以知道 $s[m][n][k]$ 就是要求的答案。

考虑边界,空空相对,0 组有一种方法,所以 $s[i][0][0]=1$,接下来考虑状态转移方程: $s[i][j][k]$ 是很容易求出来的, $s[i][j][k] = s[i-1][j][k] + f[i][j][k]$

很好理解, $s[i][j][k]$ 的方案数目就等于上一个阶段(串 **A** 还没有使用第 $i-1$ 个位置) 的方案数目,加上串 **A** 使用第 $i-1$ 个位置的方案总数。

→ 分析

怎么求解 $f[i][j][k]$ 呢？这运用了类似最长公共子序列的思想。如果 $A[i-1]=B[j-1]$ ，那么串 A 的 $i-1$ 位置就可以跟 $i-2$ 位置合成一组（如果可能的话），这时组数不变；也可以不跟 $i-2$ 位置合成一组，自成一组，这时组数+1。所以

$$f[i][j][k]=f[i-1][j-1][k]+s[i-1][j-1][k-1](A[i-1]=B[j-1])$$

如果 $A[i-1]\neq B[j-1]$ 呢？根据定义， $f[i][j][k]$ 要恰好选串 A 第 i 个字符，现在选不了，自然为 0。所以

$$f[i][j][k]=0$$

综上所述，得到完整的状态转移方程：

$$f[i][j][k]=\begin{cases} f[i-1][j-1][k]+s[i-1][j-1][k-1] & (A[i-1]=B[j-1]) \\ 0 & (A[i-1]\neq B[j-1]) \end{cases}$$

$$s[i][j][k]=s[i-1][j][k]+f[i][j][k]$$

最后数组压到二维的就可以省空间了。

The background features a central point from which several rays of varying colors (blue, orange, red, and grey) extend outwards. Faint binary code (0s and 1s) is scattered across the background, and a small line graph with two lines (one blue, one orange) is visible on the right side.

Thank you!